

Linux Kernel Development Using Git



This article looks at setting up Git and using it for Linux kernel development. It is aimed at developers who would like to move their development environment to Git SCM. I assume that readers are well versed with source-control software.

Git is a powerful source-code-control software. Its fundamental design goal was ‘high performance’. Earlier, Linux kernel development used a free version of a proprietary software called BitKeeper. Later, when BitKeeper stopped being free, Linus Torvalds came up with the idea of Git (Global information tracker). He was the initial author of the software, and later it was passed to Junio Hamano. Linux has a special need for high-performance software, due to the volume of kernel patches he handles, and Git was the answer. The major advantage of Git is that it need not be connected to a server for committing code. Git’s homepage is <http://git-scm.com/>.

Prerequisites

You need to have Git downloaded and installed on the host machine (I am using Ubuntu Maverick). It can be downloaded from <http://www.kernel.org/pub/software/scm/git/>. If you are using Ubuntu, you can simply issue the command `sudo apt-get install git-core`. Once installed, Git can be used as a regular user, and does not need super-user privileges.

Git

The `git config` command is used to set the Git repository options. The following examples are the most common options you need to set before you perform a code commit:

```
$ git config --global user.name "Surya Prabhakar"
$ git config --global user.email surya.prabhakar@del1.com
```

The `--global` option saves these options in the user’s home directory, in the file `~/.gitconfig`. If you need to manipulate a specific repository’s settings, instead of the global options, you need to `cd` into the repository directory, and issue the above commands without the `--global` switch. This will save the options in the `config` in the `.git` directory within the repository. System-wide changes can also be performed if the `--system` tag is used. For more options, refer to the man page with `man git-config`.

The Linux kernel with Git

Let’s concentrate on setting up your Linux kernel repository using Git. The Linux kernel has multiple Git repositories online. In fact, architectures like *arm* maintain their own Git repository. Here, let’s try syncing up with Linus Torvalds’ *linux-2.6* Git repository. You can obtain information about other repositories from kernel.org.

Linus Torvalds’ repo can be cloned to your local machine using the following command:

```
$ git clone git://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux-2.6.git
```

Refer to <http://git.kernel.org/?p=linux/kernel/git/torvalds/linux-2.6.git;a=summary> for more *linux-2.6* URLs. Depending on your Internet connection speed, it could take quite some time to download the whole kernel repository. Git uses port 9418 for communication with the Git server. If you are behind a corporate firewall, the above will not work. For this reason, developers have provided an alternate URL to be used over HTTP port 80:

```
$ git clone http://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux-2.6.git
```

Note: In case you have to go out via a proxy server, do not forget to set and `export` the proxy server address in the environment variable `http_proxy`, which Git uses, e.g., `export http_proxy=http://your-corporate-proxy.com/:<port>`.

The `clone` command creates a directory named *linux-2.6* in your current directory.

To update your repository, use the command `git pull`. This downloads all new commits since the last `git pull`. Once this is done, you can work on your local copy without needing an Internet connection. Use ‘`git pull`’ as follows:

```
$ cd linux-2.6
$ git pull
```

The above command will automatically use the original